

```
theory Week_2_sol
imports Main
begin
```

King's College London
Department of Informatics

Semester 1

Formal Verification: 6CCS3VER

Exercise Sheet 7

Important Note: Please download Isabelle from the link provided in the slides and bring a laptop to the large group tutorial. Performing the proofs in Isabelle will be an integral part of those tutorials.

Before you start, please open a new Isabelle/HOL theory file called `Week_1.thy` and add the following to its beginning.

```
theory Week_2
imports Main
begin
```

Exercise 7.1 Addition Commutes

Consider the function $(+)$ defined in Isabelle/HOL for natural numbers.

Prove the following theorem, first pen-and-paper, and then formally in Isabelle.

lemma *add_commutes*: “ $(n::nat) + m = m + n$ ”

Hints: 1. Perform the proof by induction on n , using substitutions *subst*, and forward reasoning using the method *rule*.

2. You will need to identify multiple lemmas about how *add* works. Do not prove those lemmas. Use *sorry* as the proof. This is the top-down approach.

3. There is a proof which uses the following lemmas:

add_0_right, *add_0_left*, *add_Suc_right*, *add_Suc*

```
proof(induction n)
  case 0
  have 1: “ $0 + m = m$ ”
```

```

    using Groups.comm_monoid_add_class.add_0[where ?a = m]
  .

have 2: "m + 0 = m"
  using Groups.monoid_add_class.add_0_right
  .

show ?case
  apply(subst 1)
  apply(subst 2)
  ..

next
case (Suc n)
have 1: "(Suc n) + m = Suc (n + m)"
  using Nat.plus_nat.add_Suc
  .

have 2: "m + Suc n = Suc (m + n)"
  using Nat.add_Suc_right
  .

show ?case
  apply(subst 1)
  apply(subst 2)
  apply(subst Suc.IH)
  ..

qed

```

Exercise 7.2 Multiplication is Monotone

Consider the multiplication function ($*$) defined in Isabelle. Prove the following property of it, i.e. that it is monotonically increasing in both arguments. You should prove it both, pen-and-paper and in Isabelle.

lemma *mult_mono*:

fixes $a\ b\ c\ d::nat$ — Note the alternative way of fixing the types of the constants
assumes " $a \leq b$ " " $c \leq d$ "
shows " $a * c \leq b * d$ "

Your pen-and-paper proof should indicate

1. what lemmas are you assuming,
2. on what variable are you performing induction,
3. what are assumptions and the proof goals in the base case and the step case, and what is the induction hypothesis, and

4. how are you instantiating the induction hypothesis, i.e. how do you show that its assumptions hold and which quantified variables are instantiated with which constants.

Hints: 1. The proof should be by induction

2. The following lemmas suffice to prove the goal with only substitution, forward reasoning, and induction:

refl, *diff_le_self*, *mult_le_mono2*, *mult_zero_right*, *le0*, *mult_Suc_right*, *add_mono*, *Suc_eq_plus1*, *le_diff_conv*

3. When you have a term like $x - y$, where x is a *nat*, in the goal or the assumptions, you should perform proof by case analysis on whether $y \leq x$. This is because of the way $-$ is defined for natural numbers, where $x - y = 0$, for any $x \leq y$.

```

using assms
proof(induction d)
  case assms: 0
  have 1: “ $c = 0$ ”
    using Nat.bot_nat_0.extremum_uniqueI[OF assms(2)]
    .
  find_theorems “ $?x \leq ?x$ ”

  show ?case
    apply(subst 1)
    apply(subst mult_zero_right)
    apply(subst mult_zero_right)
    using preorder_class.order.refl
    .

next
  case Suc: (Suc d)

  show ?case
  proof(cases “ $c = \text{Suc } d$ ”)
    case c_eq_sucd: True
    show ?thesis
      apply(subst c_eq_sucd)
      using mult_le_mono1[OF assms(1)]
      .
    .

  next
  case c_lt_sucd: False

  have 1: “ $c > \text{Suc } d \vee c < \text{Suc } d$ ”
    apply(subst neq_iff[symmetric])
    using c_lt_sucd[symmetric]
    .
  have 2: “ $\neg c > \text{Suc } d$ ”
    using leD[OF Suc(3)]
    .
  have 3: “ $c < \text{Suc } d$ ”

```

```

    using Meson.make_pos_rule'[OF 1 2]
    .

have 4: " $c \leq d$ "
  apply(subst less_Suc_eq_le[symmetric])
  using 3
  .

have 5: " $a * c \leq b * d$ "
  using Suc.IH[OF assms(1) 4]
  .

have 6: " $b * d \leq b * (Suc\ d)$ "
  apply(rule mult_le_mono2)
  apply(rule order.strict_implies_order)
  using Nat.lessI
  .
show ?thesis
  using le_trans[OF 5 6]
  .

qed

qed

end

```